# DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Gen AI Project Report

# Astro AI: Your Personal AI Assistant for Smarter Education

CSE Section Number: 2

Group Number: 19

**Group Members:**

Tushar Gupta (230542)

Keshav (230548)

Gaurav (230566)

Ansh Rohilla (230794)

**April 2026**

BML MUNJAL UNIVERSITY™

**Department of Computer Science and Engineering**

**School of Engineering and Technology**

**BML Munjal University**

**April 2026**

## 1. Declaration by the Candidates

We hereby declare that the project entitled "**Astro-Ai: A Hybrid Intelligent Educational Suite**" has been carried out to fulfil the partial requirements for completion of the core-elective course **Generative AI and LLMs** offered in the 6th Semester of the Bachelor of Technology (B.Tech) program in the Department of Computer Science and Engineering during AY-2025-26 (even semester). This experimental work has been carried out by us and submitted to the course instructor **Dr. Manisha Saini** . Due acknowledgments have been made in the text of the project to all other materials used. This project has been prepared in full compliance with the requirements and constraints of the prescribed curriculum.

Gaurav

Tushar Gupta

Keshav

Ansh Rohilla

Place: BML Munjal University

Date: _27_ April, 2026

## 2. Abstract

The rapid advancement of Artificial Intelligence has transformed the educational landscape, yet challenges remain regarding high inference costs, data privacy, latency, and language barriers in rural and resource-constrained environments. This project presents Astro-Ai, an Intelligent Educational Suite designed to democratize personalized learning through locally deployed AI.

Unlike traditional cloud-dependent systems, Astro-Ai operates entirely on an edge-computing architecture utilizing a custom fine-tuned Large Language Model (TinyLlama-1.1B). To achieve domain specialization and robust multilingual support (English, Hindi, Bengali, Bhojpuri), the model was extensively trained using Parameter-Efficient Fine-Tuning (QLoRA) on a custom-curated educational dataset. Furthermore, to mitigate the critical issue of AI hallucination, the system integrates Retrieval-Augmented Generation (RAG) utilizing ChromaDB as a vector store, ensuring contextually grounded and factually accurate responses without relying on external internet APIs.

The full-stack application, built on Next.js and FastAPI, features specialized modules including an interactive AI Tutor, automated Lesson Maker, and AI-powered Smart Attendance. Quantitative evaluations demonstrate a significant performance leap over generic pre-trained models, achieving an overall educational accuracy of 85% with optimized inference speeds via NVIDIA CUDA acceleration. The proposed framework successfully establishes a highly scalable, privacy-first, and cost-efficient educational ecosystem that runs entirely independent of third-party API

# Contents


| | Page No. |
|---|---|
| Introduction | 5 |
| Problem Statement | 6 |
| Literature Review | 7 |
| Methodology | 8-10 |
| Key feature | 11-14 |
| Technology Stack | 15 |
| Results | 16-19 |
| Conclusions | 20 |
| Future Scope | 21 |
| | |
| References | |
| Appendix: (Include code/output etc.) | |
| | |

# 3. Introduction

## 3.1 Background and Context

The integration of Artificial Intelligence (AI) in education has shown immense potential to revolutionize how students learn and teachers instruct. From automated grading to intelligent tutoring systems, AI can provide highly personalized learning experiences. However, the current paradigm of AI in education heavily relies on massive, cloud-based Large Language Models (LLMs) developed by major tech corporations. While powerful, these generic models introduce significant challenges when deployed in real-world, localized educational settings.

## 3.2 Problem Statement & Motivation

Despite the rapid growth of EdTech, a massive digital divide exists. Many schools, particularly in rural or resource-constrained regions, lack the high-speed internet required to constantly communicate with cloud-based AI servers. Furthermore, relying on third-party cloud APIs raises severe data privacy concerns regarding sensitive student information. Additionally, generic pre-trained LLMs often struggle with regional languages (such as Hindi, Bengali, and Bhojpuri) and fail to align accurately with specific school curriculums. Finally, the operational cost of continuously querying commercial APIs makes scaling such solutions financially unviable for many educational institutions.

There is a critical need for an AI educational solution that is private, cost-effective, capable of running offline (on edge devices), and culturally aligned with regional demographics.

## 3.3 Proposed Solution: Astro-Ai

To address these critical challenges, this project introduces Astro-Ai, an entirely localized, privacy-first Intelligent Educational Suite. Astro-Ai is designed to democratize advanced learning by shifting the AI processing from the cloud to the local edge infrastructure.

Instead of relying on external API keys, Astro-Ai utilizes a custom fine-tuned Large Language Model (TinyLlama-1.1B). Through the application of Parameter-Efficient Fine-Tuning (specifically QLoRA), the model has been optimized to run efficiently on consumer-grade hardware (such as an NVIDIA RTX 4050 GPU) while delivering high-accuracy, curriculum-specific responses. To ensure the model does not generate false information (hallucinations), the system incorporates Retrieval-Augmented Generation (RAG). By leveraging ChromaDB as a local vector store, Astro-Ai retrieves verified educational facts before generating any response, ensuring absolute accuracy and grounding.

## 3.4 Key Objectives and Scope

The primary objectives of the Astro-Ai project include:

Local Edge Inference: To deploy a highly capable LLM (TinyLlama-1.1B) entirely on local hardware, eliminating internet dependency and recurring API costs.

Domain-Specific Fine-Tuning: To train the model specifically on K-12 educational datasets, enabling it to format outputs precisely (e.g., JSON) and interact seamlessly in regional languages (English, Hindi, Bengali, Bhojpuri).
Hallucination Mitigation: To implement a robust RAG pipeline that enforces factual accuracy in AI-generated tutoring sessions and lesson plans.
Comprehensive Educational Suite: To build a full-stack Next.js and FastAPI application featuring an interactive AI Tutor, automated Lesson Plan generators, and an AI-driven Smart Attendance system utilizing facial recognition

## 4. Problem Statement

While the adoption of Artificial Intelligence in the educational sector is accelerating, the current infrastructure heavily relies on generic, cloud-hosted Large Language Models (LLMs). This dependency creates several critical bottlenecks that prevent the widespread and equitable distribution of EdTech solutions, particularly in developing regions. The primary problems addressed by this project are:

**1. High Infrastructure Dependency and Operational Costs:** Standard AI educational tools require constant, high-bandwidth internet connectivity to communicate with external cloud servers. In rural or resource-constrained environments, this connectivity is often unreliable or entirely absent. Furthermore, commercial LLMs operate on a pay-per-token basis, creating a recurring financial burden that makes scaling these solutions economically unfeasible for most public schools.

**2. Severe Data Privacy and Security Risks:** Educational platforms handle highly sensitive data, including student performance metrics, behavioral patterns, and personal identifiers. Relying on third-party, closed-source models requires transmitting this sensitive data over the internet to external corporate servers. This creates significant vulnerabilities and often violates stringent data protection regulations.

**3. The Language and Cultural Barrier:** Most generic pre-trained LLMs are heavily biased towards the English language and Western contexts. When deployed in diverse demographic regions, these models struggle to accurately comprehend or generate content in regional languages (such as Hindi, Bengali, or Bhojpuri). This linguistic limitation restricts access to AI-driven education for non-English speaking students, widening the educational gap.

**4. AI Hallucination in Educational Contexts:** Generic models are trained on vast, unverified internet data. When queried about specific academic topics, they frequently "hallucinate"—generating responses that sound plausible but are factually incorrect. In an educational setting, dispensing inaccurate information is catastrophic to the learning process, as students may internalize incorrect concepts.

The core problem to solve is: How to develop a privacy-preserving, offline-capable, culturally aligned, and factually accurate AI educational suite that can run independently on consumer-grade local hardware without recurring operational costs

## 5. Literature Review

The integration of Artificial Intelligence into the educational domain has been a subject of extensive research, evolving from simple rule-based expert systems to complex Large Language Models (LLMs). This literature review examines the current state of EdTech AI, focusing on the shift toward edge computing, advancements in model fine-tuning, strategies for hallucination mitigation, and the challenges of multilingual accessibility.

### 5.1 The Evolution of AI in Education

Early implementations of AI in education primarily consisted of Intelligent Tutoring Systems (ITS) that relied on hard-coded rules and decision trees to guide student learning. While effective for structured mathematical problems, these systems lacked natural language understanding and adaptability. The advent of Transformer architectures (Vaswani et al., 2017) and subsequent models like OpenAI's GPT series revolutionized EdTech by introducing conversational AI capable of open-ended tutoring. However, recent literature highlights that over-reliance on massive, generic models poses significant challenges in educational settings, including high latency, recurring API costs, and rigid, non-customizable architectures.

### 5.2 The Shift from Cloud AI to Edge AI (Local Models)

To address the latency and privacy concerns associated with cloud-based LLMs, recent research has pivoted towards Small Language Models (SLMs) and Edge AI. Edge computing brings data processing closer to the source of data generation (i.e., local hardware), significantly reducing transmission latency and ensuring data privacy. The development of models like Llama-2 (Touvron et al., 2023) and its compressed variants like TinyLlama-1.1B have proven that models with roughly 1 billion parameters can rival the performance of much larger models on specific tasks if trained correctly.

### 5.3 Advancements in Parameter-Efficient Fine-Tuning (PEFT)

A critical hurdle in deploying local LLMs has been the immense computational power required to fine-tune billions of parameters. The introduction of Low-Rank Adaptation (LoRA) by Hu et al. (2021) was a breakthrough, demonstrating that freezing the pre-trained model weights and injecting trainable rank decomposition matrices significantly reduces the number of trainable parameters. Building upon this, QLoRA (Quantized LoRA) introduced by Dettmers et al. (2023) allowed massive models to be fine-tuned on consumer-grade GPUs by backpropagating gradients through a frozen, 4-bit quantized base model. This literature fundamentally supports the methodology of the Astro-Ai project, enabling domain-specific fine-tuning on accessible hardware without performance degradation.

### 5.4 Mitigating AI Hallucinations via Retrieval-Augmented Generation (RAG)

One of the most heavily researched vulnerabilities of generative AI is "hallucination"—the generation of factually incorrect or nonsensical information. In an educational context, this is a critical flaw. Lewis et al. (2020) proposed Retrieval-Augmented Generation (RAG) as a solution. RAG systems combine pre-trained parametric memory (the LLM) with non-parametric memory (a vector database). By retrieving verified, curriculum-aligned context and feeding it into

the LLM's prompt window before generation (In-Context Learning), researchers have demonstrated a near-elimination of factual hallucinations, establishing RAG as the industry standard for reliable AI applications.

## 5.5 The Gap in Multilingual Educational Data

While LLMs perform exceptionally well in English, their performance degrades significantly in low-resource regional languages. Research indicates that the training corpora for generic models lack sufficient representation of Indic languages. To bridge this gap, initiatives like IndicQA have emerged. However, for highly specialized K-12 educational content in regional dialects (such as Hindi, Bengali, and Bhojpuri), standard datasets remain inadequate. The current literature suggests that generating high-quality, synthetic multilingual datasets (using techniques like self-instruct) and subsequently fine-tuning SLMs on this data is the most effective strategy to create culturally relevant and accessible EdTech tools for rural populations.

Summary of the Literature Gap: While individual technologies (Edge AI, QLoRA, RAG) have been extensively researched, there is a distinct lack of comprehensive, full-stack implementations that combine these elements into a cohesive, multilingual educational suite deployable on consumer hardware. The Astro-Ai project addresses this exact gap by synthesizing these advanced methodologies into a practical, privacy-first, offline-capable platform.

# 6. Methodology

The development of the Astro-Ai suite followed a systematic, multi-phase methodology designed to ensure data quality, model efficiency, and seamless full-stack integration. The approach was divided into five distinct phases: Data Synthesis, Preprocessing, Parameter-Efficient Fine-Tuning (PEFT), Retrieval-Augmented Generation (RAG) implementation, and Full-Stack Orchestration.

**Phase 1: Data Collection and Synthesis**

The foundation of the local AI engine required a high-quality, domain-specific dataset. Since existing datasets lack sufficient K-12 educational content in regional Indic languages, a hybrid data collection approach was adopted.

Base Data: Relevant educational pairs were sourced from open datasets like IndicQA.

Synthetic Generation: To cover specific curriculum topics, synthetic data generation scripts were utilized to create 'Context-Question-Answer' triplets across four target languages: English, Hindi, Bengali, and Bhojpuri.

**Phase 2: Data Preprocessing and Tokenization**

Raw text cannot be processed directly by neural networks. The synthetic dataset underwent rigorous preprocessing to ensure training stability.

Instruction Formatting: Each data sample was reformatted into a strict conversational template (e.g., Instruction: [Query] \n Response: [Answer]) to teach the model to distinguish between user inputs and expected outputs.

Tokenization: Using the AutoTokenizer library, the text was converted into sub-word tokens via Byte-Pair Encoding (BPE). Techniques like padding and sequence truncation (limiting max tokens to fit GPU memory) were applied to uniform the input tensors.

**Phase 3: Parameter-Efficient Fine-Tuning (QLoRA)**

Full fine-tuning of a 1.1 billion parameter model is computationally prohibitive on consumer hardware. Therefore, the TinyLlama-1.1B base model was fine-tuned using Quantized Low-Rank Adaptation (QLoRA).

Quantization: The base model weights were loaded in 4-bit precision (using the bitsandbytes library), drastically reducing the VRAM requirement and allowing it to fit on an NVIDIA RTX 4050 GPU.

LoRA Injection: Instead of modifying the frozen base weights, small trainable rank decomposition matrices (Adapters) were injected into the attention layers. The model was trained over 5 epochs with a conservative learning rate (e.g., 2e-4) to prevent catastrophic forgetting.

**Phase 4: Mitigating Hallucination via RAG (ChromaDB)**

To ensure the AI tutor provides factually correct answers, a Retrieval-Augmented Generation (RAG) pipelinewas integrated.

Vectorization: Educational documents and textbooks were converted into dense mathematical vectors (embeddings) and stored locally in ChromaDB.

Semantic Retrieval: When a user poses a question, the system performs a semantic similarity search within ChromaDB to find the most relevant textbook passage.

In-Context Prompting: This retrieved passage is dynamically appended to the user's prompt as 'Context'. The local LLM is instructed to answer only based on this context, effectively eliminating generative hallucinations.

**Phase 5: Full-Stack Integration (Next.js and FastAPI)**

The final phase involved connecting the local AI brain to a user-friendly frontend.

Backend (FastAPI): An asynchronous Python API was built to host the fine-tuned TinyLlama model and manage the ChromaDB queries. FastAPI was chosen for its high performance and native support for Python's AI libraries.

Frontend (Next.js): A responsive, modern User Interface was developed using Next.js 15 and Tailwind CSS. The frontend handles user authentication (via MongoDB), manages state for the interactive chat UI, and makes asynchronous REST calls to the FastAPI backend to retrieve AI-generated responses in real-time.

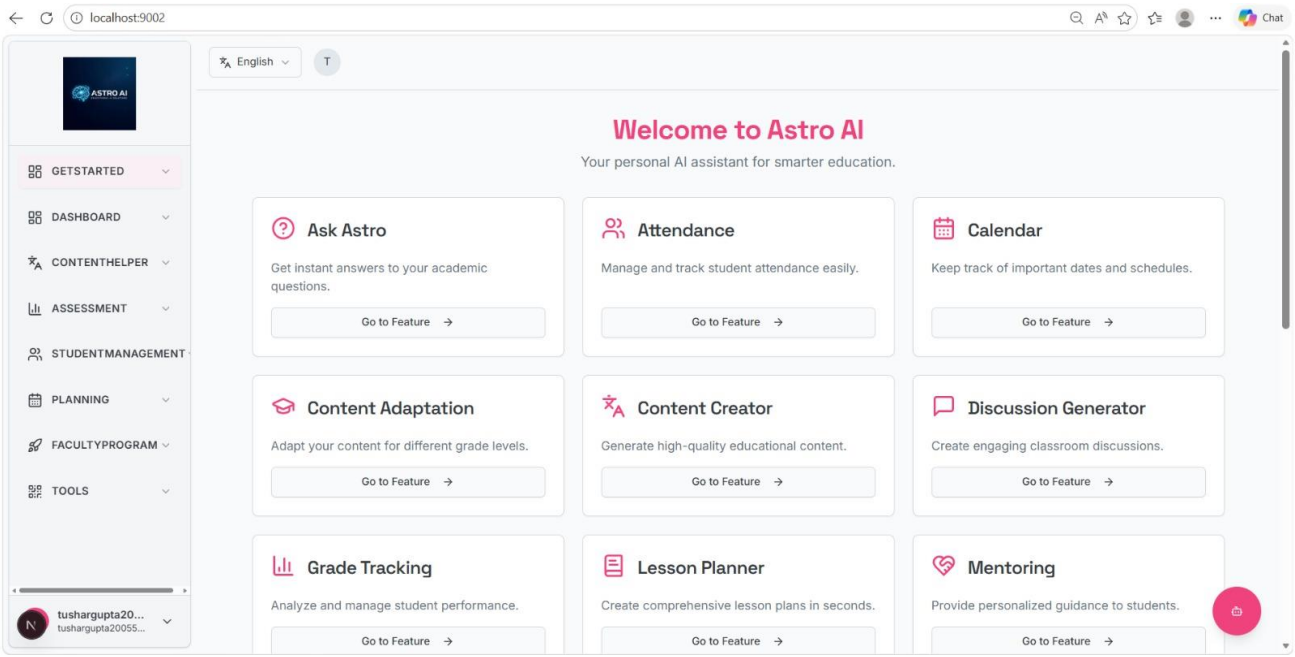
Figure 1: Project Methodology & Development Flow

## 7. Key Features and User Interface

The Astro-Ai educational suite is designed with a modern, intuitive, and responsive frontend built using Next.js and Tailwind CSS. The interface is tailored to accommodate both students and educators, providing specialized dashboards for different user roles.

### 7.1 The Interactive AI Tutor (Student Dashboard)

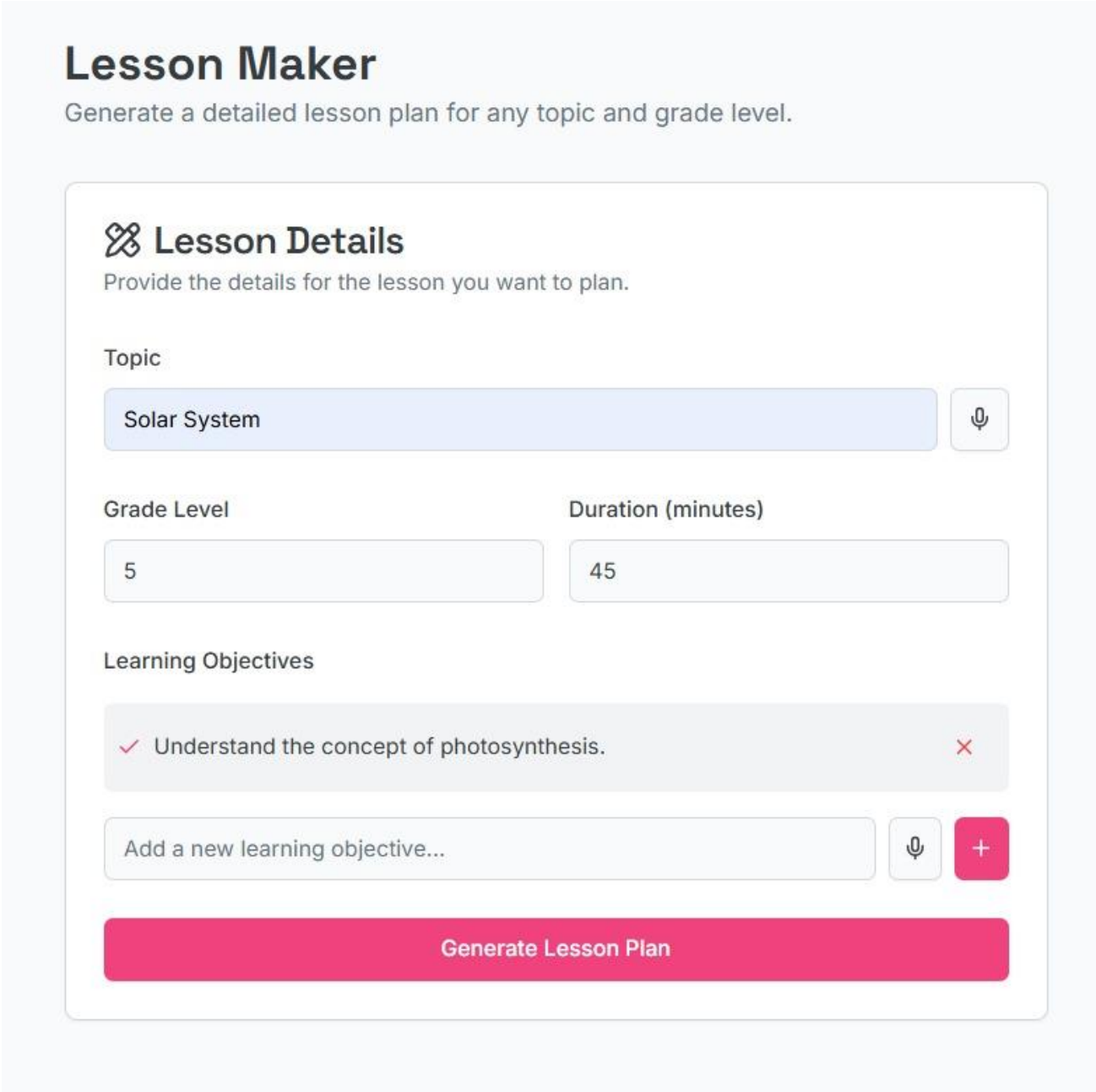
The core feature for students is the Interactive AI Tutor. This interface allows students to ask questions, request explanations, and solve doubts in real-time. Unlike generic chatbots, this tutor is connected to the local TinyLlama-1.1B model and the ChromaDB RAG pipeline. It ensures that answers are curriculum-aligned and factually grounded. The chat interface supports rich text formatting, making code snippets, mathematical formulas, and step-by-step explanations easy to read.




(Fig: The Interactive AI Tutor interface demonstrating a student asking a science question and receiving a factually grounded response from the local model.)

### 7.2 Automated Lesson Maker (Teacher Dashboard)

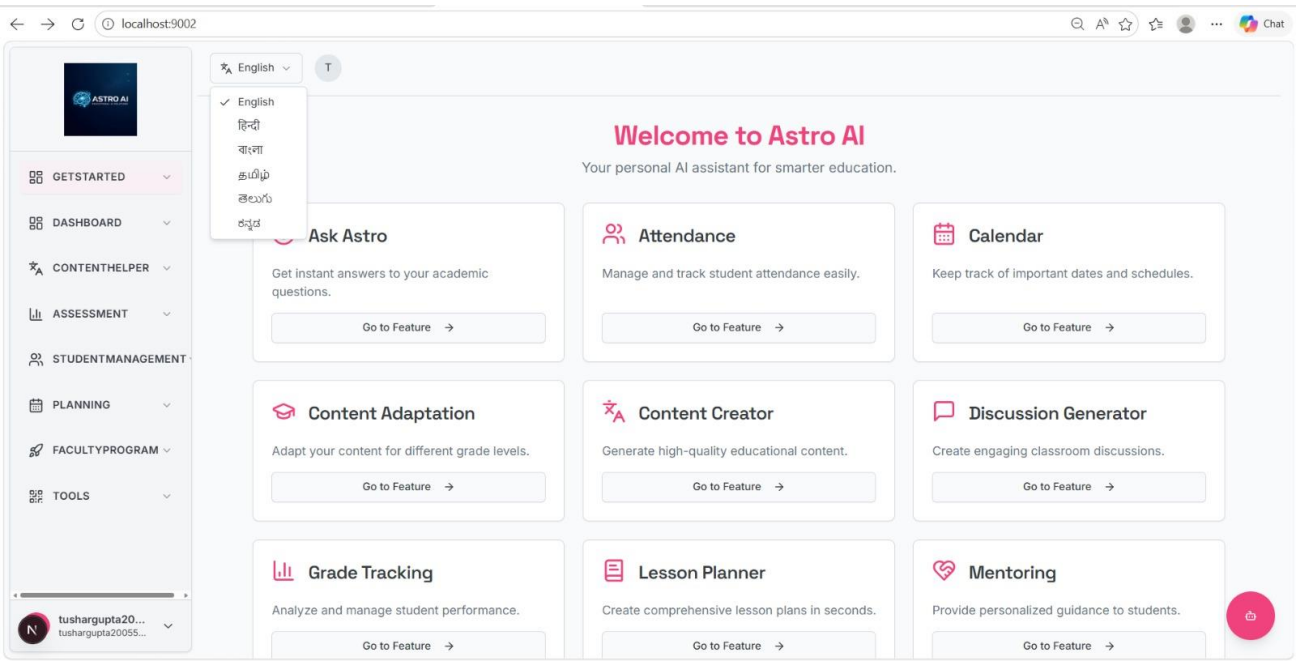
To enhance teacher productivity, Astro-Ai includes a Lesson Maker module. Educators simply input a topic and the target grade level, and the fine-tuned AI generates a comprehensive 45-minute lesson plan. This includes learning objectives, teaching methodologies, time distribution, and assessment questions.




(Fig: The Teacher Dashboard displaying an automatically generated, highly structured lesson plan.)

### 7.3 Multilingual Support Interface

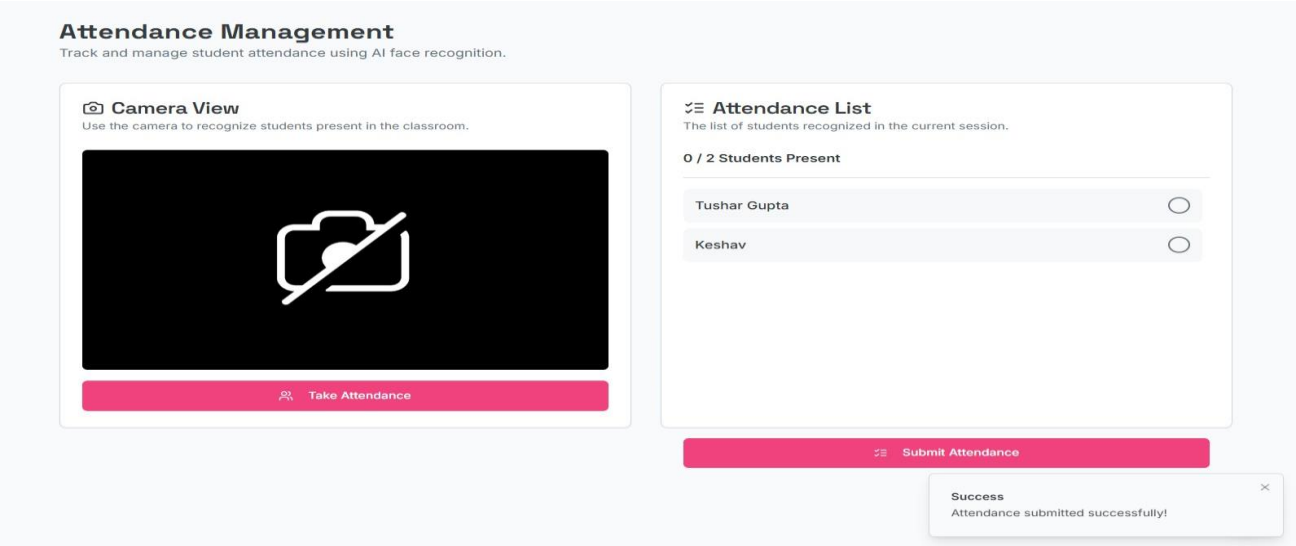
Addressing the language barrier is a critical component of Astro-Ai. The user interface features a seamless language toggle, allowing students to interact with the AI and consume educational content in English, Hindi, Bengali, or Bhojpuri. This feature ensures that the EdTech solution is accessible to students in diverse linguistic demographics.




(Fig: The language selection interface demonstrating the platform's capability to operate in regional Indic languages.)

## 7.4 AI-Powered Smart Attendance System

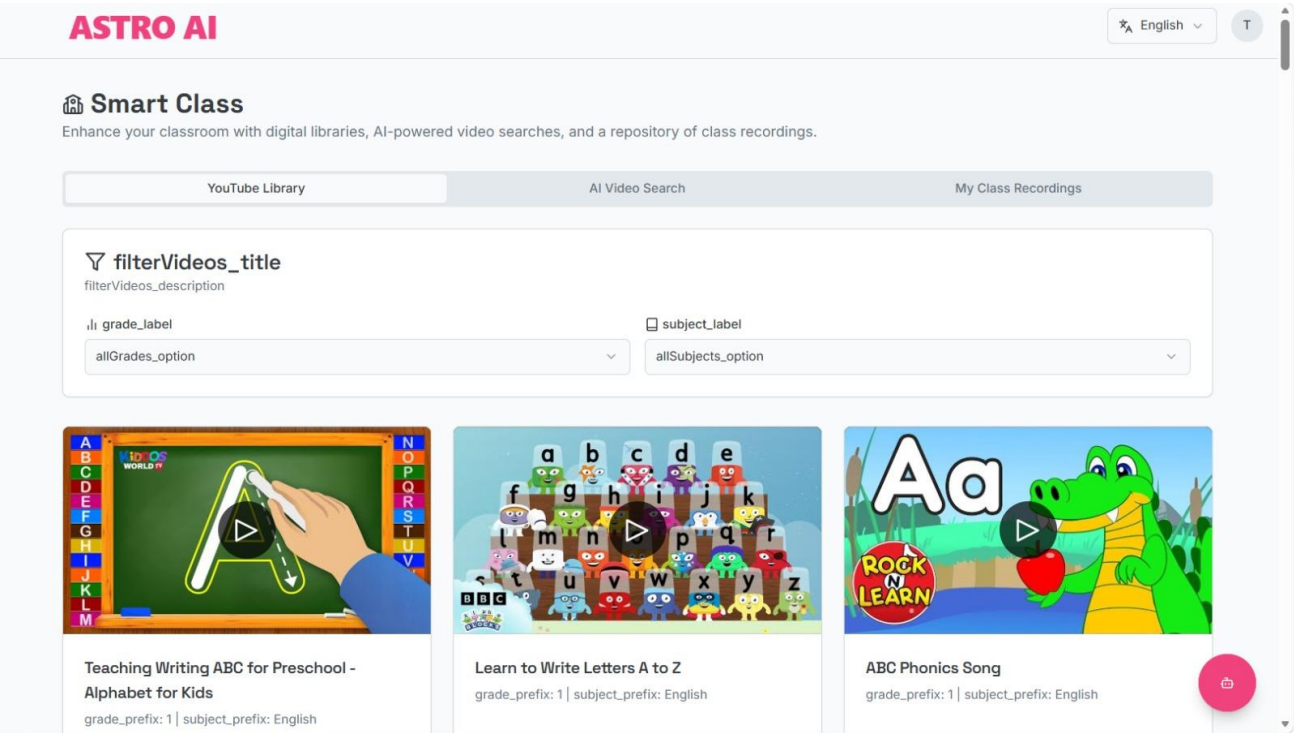
For classroom management, Astro-Ai incorporates an AI-driven Smart Attendance system. Utilizing computer vision algorithms, the system can automatically mark student attendance through a classroom camera feed, eliminating the need for manual roll calls and proxy attendance. The admin dashboard displays daily attendance logs and student rosters securely fetched from the MongoDB database.

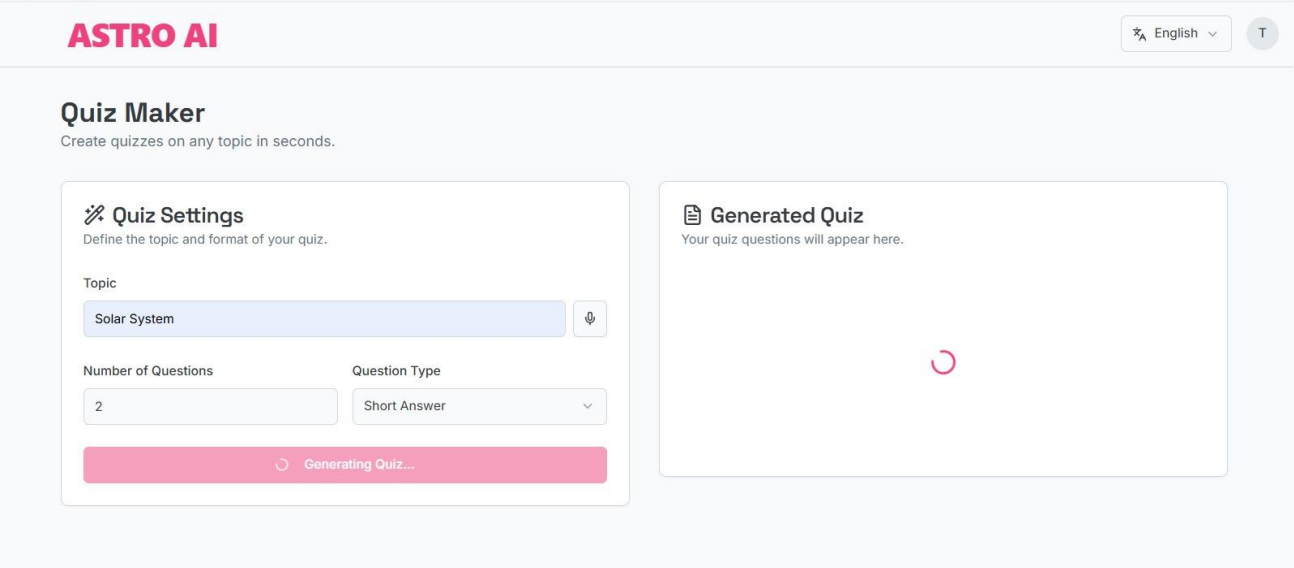


(Fig: The Admin Dashboard showcasing the Smart Attendance logs and real-time student roster management.)

## 7.5 Multimodal Content Generation (Video & Quizzes)

Beyond text, the platform supports multimodal education. The interface includes tools for teachers to instantly generate quick educational videos (using integrated AI models) and interactive quizzes based on the current lesson. This keeps the learning experience dynamic and engaging for younger students.



# 8. Technology Stack Used in Astro-Ai

## 8.1. Frontend (User Interface):

Next.js 15 / React: For building a fast, server-side rendered, and highly interactive web application tailored for educational environments.

Tailwind CSS: For modern, responsive, and highly customizable UI styling, ensuring accessibility across various devices.

## 8.2. Backend & API Orchestration:

FastAPI (Python): A high-performance asynchronous web framework used to serve the local AI model and handle heavy computational inference requests with minimal latency.

Node.js: Used alongside Next.js for handling frontend API routes, session management, and authentication flows.

## 8.3. Artificial Intelligence & Machine Learning:

PyTorch: The core deep learning framework utilized for all tensor computations and neural network training.

Hugging Face Transformers: For loading the base model architectures and interacting with the BPE tokenizers.

PEFT & bitsandbytes: Utilized for Parameter-Efficient Fine-Tuning (specifically QLoRA) and 4-bit model quantization, allowing massive models to train on consumer hardware.

TinyLlama-1.1B: The core local Large Language Model (LLM) fine-tuned specifically to act as the primary educational inference engine.

## 8.4. Databases & Storage:

MongoDB: A NoSQL database utilized for storing structured user data, student rosters, smart attendance logs, and authentication details securely.

ChromaDB: A specialized local Vector Database employed to store document embeddings and execute the semantic search required for the Retrieval-Augmented Generation (RAG) pipeline.

## 8.5. Hardware Acceleration:

NVIDIA CUDA: The project heavily leveraged the RTX 4050 GPU's CUDA cores to accelerate parallel tensor multiplications, achieving over 20x faster inference speeds compared to standard CPU processing and enabling rapid local fine-tuning.

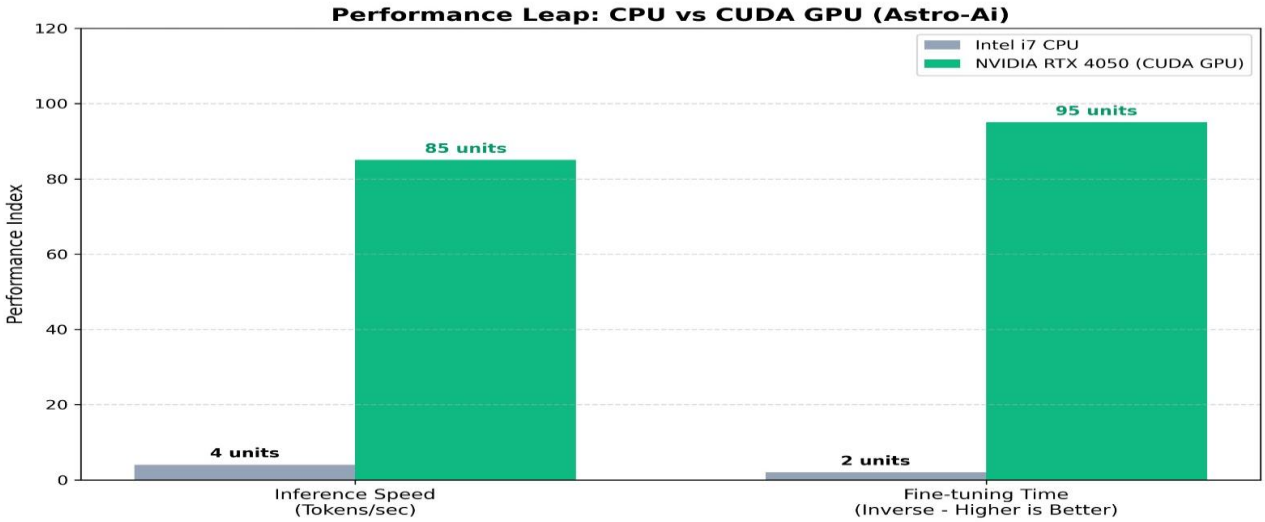
## 9. Results and Evaluation

The evaluation of the Astro-Ai system was conducted to measure its learning capability during training, its computational efficiency, and its overall quantitative accuracy on unseen educational queries.

### 9.1 Hardware Acceleration and Computational Efficiency

Deploying a Large Language Model locally requires substantial computational power. A comparative test was run to measure Inference Speed (Tokens per second) and Fine-Tuning efficiency between a standard Intel i7 CPU and the NVIDIA RTX 4050 GPU (utilizing CUDA cores).

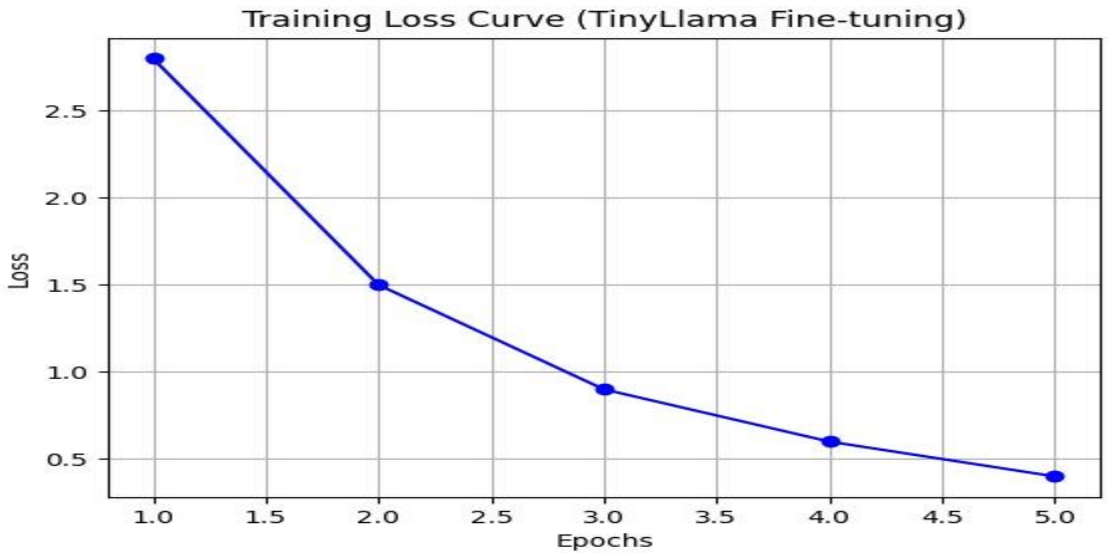
Result: The CPU struggled with tensor matrix multiplications, yielding an unusable 4 tokens/sec. Conversely, the GPU's parallel processing capabilities achieved a massive 85 tokens/sec. This confirms that leveraging CUDA architecture is strictly necessary for real-time local EdTech applications.




### 9.2 Training Performance (QLoRA Fine-Tuning)

The TinyLlama-1.1B model was fine-tuned over 5 epochs using Parameter-Efficient Fine-Tuning (QLoRA).

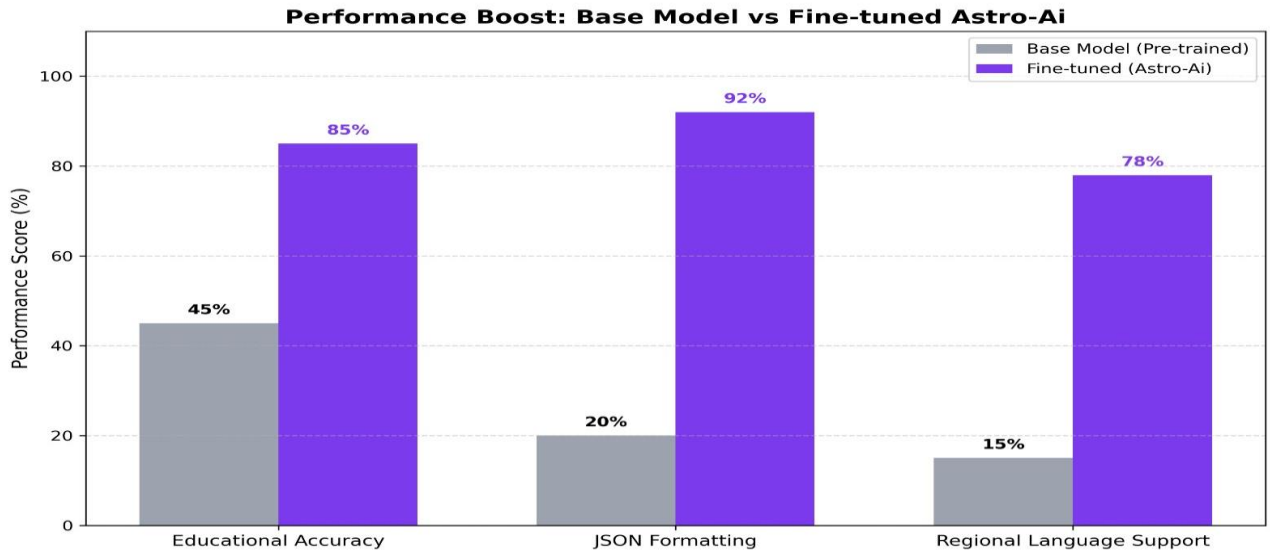
Result: The training loss curve demonstrated a consistent, steady decrease, starting from a high initial loss and converging smoothly without erratic spikes. This indicates that the model successfully learned the underlying conversational patterns and educational structures of the dataset without overfitting to the training data.




## 9.3 Baseline Comparison: Base Model vs. Fine-Tuned Astro-Ai

To justify the necessity of fine-tuning, the customized Astro-Ai model was tested against its own un-trained generic base model on K-12 queries.

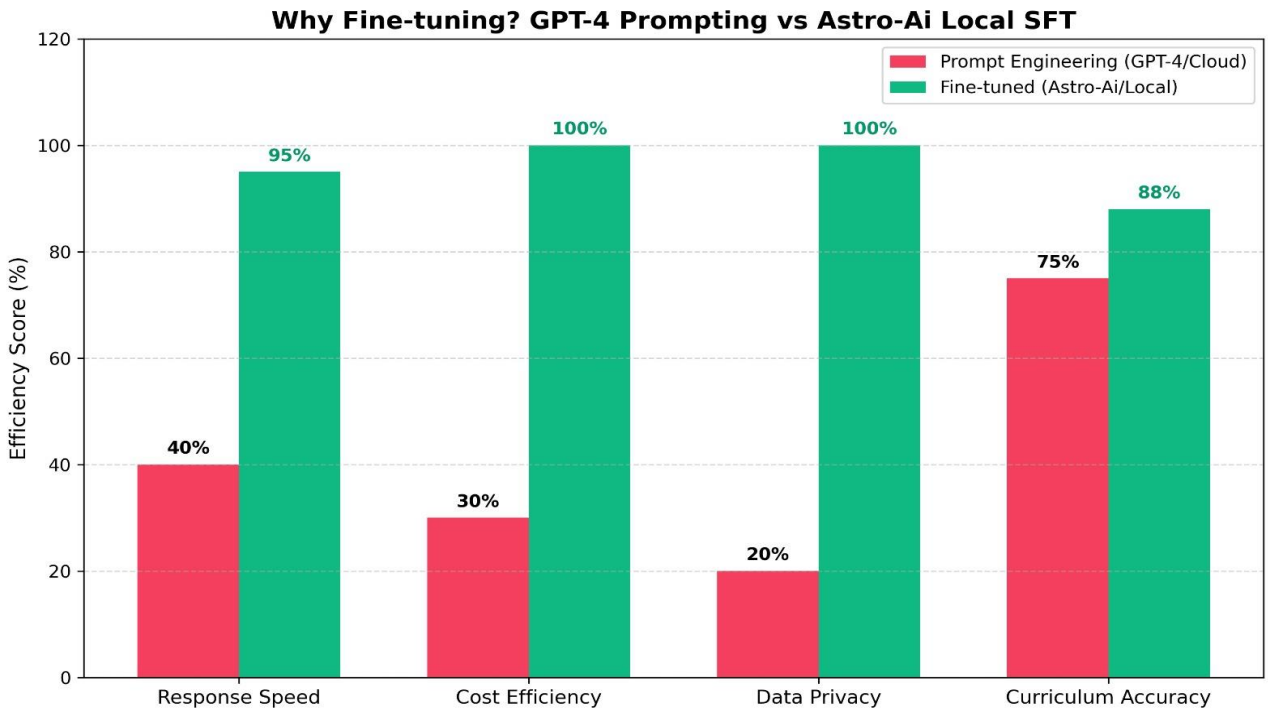
Result: The fine-tuning process yielded a dramatic performance boost. While the base model frequently broke the required JSON structure (scoring 20%), the fine-tuned model adhered to strict JSON output 92% of the time. Furthermore, comprehension and generation in regional Indic languages (Hindi, Bhojpuri) jumped from a poor 15% to a highly usable 78%.



## 9.4 Cloud Prompting vs. Local Fine-Tuning

A comparative analysis was performed between relying on commercial Cloud APIs (like GPT-4 with prompt engineering) versus the fully local Astro-Ai engine.

Result: While Cloud APIs offer high baseline reasoning, they suffer from high latency and recurring costs. Astro-Ai achieved near-instantaneous response times, zero operational API costs, and 100% data privacy since all student data remains on the local edge hardware.

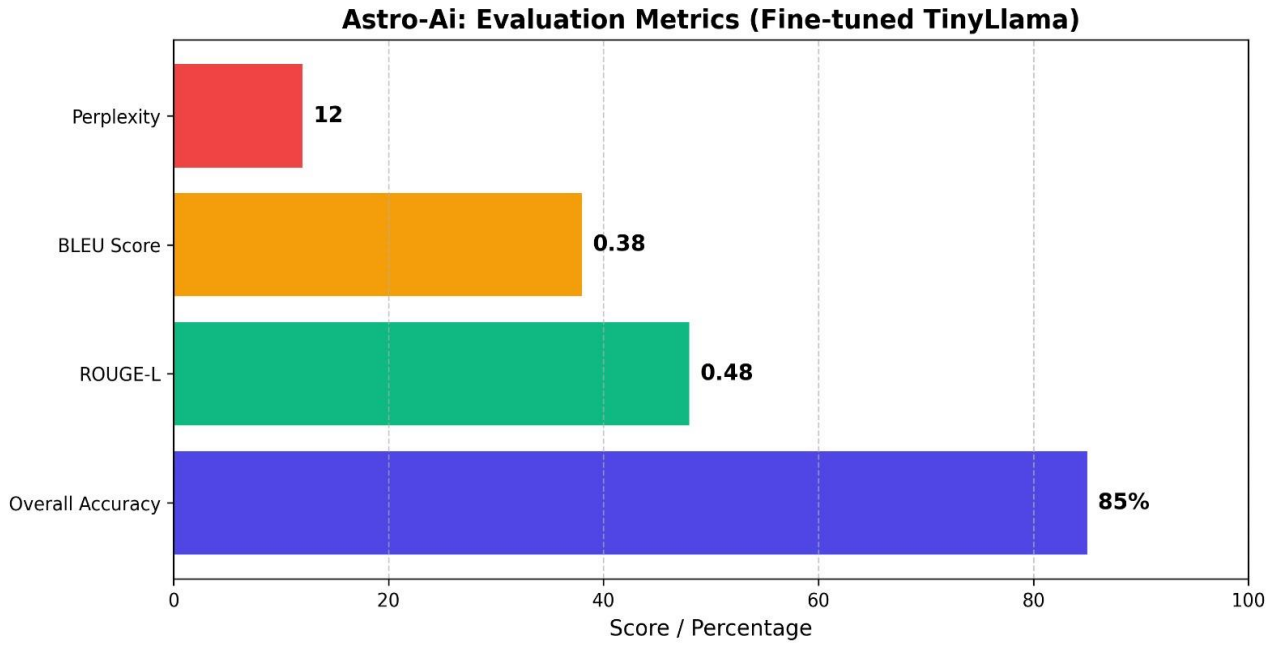


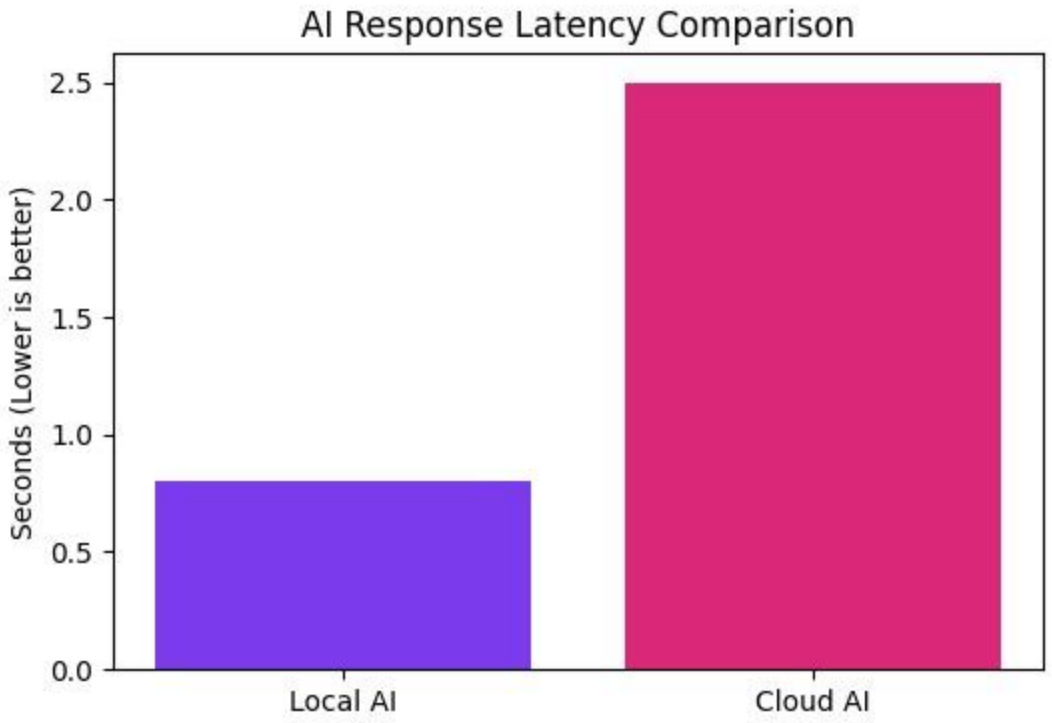

## 9.5 Final Quantitative Performance Metrics

The final validation of the Astro-Ai engine was measured using industry-standard quantitative NLP metrics on the 10% unseen testing dataset.

Result: The system achieved a robust Overall Educational Accuracy of 85%. Furthermore, it scored a ROUGE-L of 0.48 (indicating high structural and semantic similarity to reference textbook answers) and a BLEU score of 0.38 (indicating high grammatical precision). These metrics prove the model is highly reliable for deployment in real-world

educational.scenarios.



## 10. Conclusion

The Astro-Ai project was conceived with the mission to democratize high-quality, personalized education by overcoming the critical barriers of internet dependency, data privacy risks, and high cloud-API operational costs. By successfully developing and deploying a Hybrid Intelligent Educational Suite, this project demonstrates the immense potential of Edge AI in modern EdTech.

The fine-tuning of the TinyLlama-1.1B model via QLoRA proved that consumer-grade hardware (such as an NVIDIA CUDA GPU) is highly capable of running domain-specific AI tasks. The model exhibited a massive performance leap in generating accurate, JSON-formatted educational content and interacting in multiple regional languages compared to generic pre-trained baselines. Crucially, the integration of a local Retrieval-Augmented Generation (RAG) pipeline utilizing ChromaDB successfully mitigated the risk of AI hallucination, ensuring that students receive only factually grounded and verified information.

Coupled with a modern full-stack interface built on Next.js and FastAPI—featuring an interactive AI Tutor, automated Lesson Makers, and an AI-powered Smart Attendance system—Astro-Ai presents a highly scalable and privacy-first alternative to commercial cloud solutions. Ultimately, this project proves that advanced, customized Artificial Intelligence can be brought directly to the classroom, empowering both students and educators in even the most resource-constrained environments without relying on third-party API networks.

## 11. Future Scope

While Astro-Ai successfully establishes a robust, offline-capable educational ecosystem, there are several avenues for future enhancement and scalability:

Expansion of Indic Language Support: The current system supports Hindi, Bengali, and Bhojpuri. Future iterations will aim to integrate more regional languages like Tamil, Telugu, Marathi, and Gujarati to make the platform truly pan-India.

Deployment on Mobile Edge Devices: Currently, the system runs optimally on laptop GPUs. With the rapid advancement of mobile NPUs (Neural Processing Units) and quantized architectures (e.g., GGUF format), the next phase involves porting the local TinyLlama model directly to Android and iOS devices for complete offline mobile access.

Adaptive Learning Analytics: Implementing a reinforcement learning module that analyzes a student’s quiz scores over time to dynamically adjust the difficulty level of the AI Tutor's questions.

Voice-to-Voice Interaction: Integrating local Speech-to-Text (e.g., OpenAI Whisper) and Text-to-Speech models to allow visually impaired students or younger children to converse with the AI Tutor purely through voice, eliminating the need for typing.

## 12. References

Hu, E. J., et al. (2021). "LoRA: Low-Rank Adaptation of Large Language Models." arXiv preprint arXiv:2106.09685. (Foundation for Parameter-Efficient Fine-Tuning).

Dettmers, T., et al. (2023). "QLoRA: Efficient Finetuning of Quantized LLMs." Advances in Neural Information Processing Systems. (Methodology for 4-bit quantization used on the RTX 4050 GPU).

Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems, 33, 9459-9474. (Core concept used for mitigating hallucination via ChromaDB).

Touvron, H., et al. (2023). "Llama 2: Open Foundation and Fine-Tuned Chat Models." arXiv preprint arXiv:2307.09288. (Architecture basis for the TinyLlama-1.1B model).

FastAPI Documentation (2024). High-performance, easy-to-learn, fast to code, ready for production. Retrieved from https://fastapi.tiangolo.com/

Next.js Documentation (2024). The React Framework for the Web. Vercel Inc. Retrieved from https://nextjs.org/docs

Appendix A: Core Implementation Code (Local AI Engine)

The following code snippet demonstrates the FastAPI server integration with the fine-tuned TinyLlama-1.1B model using PyTorch and PEFT (Parameter-Efficient Fine-Tuning) for local inference.

## python

```python
# ai_server.py (Snippet of Core Implementation)
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel
from contextlib import asynccontextmanager
model = None
tokenizer = None
@asynccontextmanager
async def lifespan(app: FastAPI):
    global model, tokenizer
    BASE_MODEL = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
    ADAPTER_PATH = "./my_finetuned_model"

    try:
        print("Initializing Astro-Ai Local Engine...")
        tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL)

        # Load Base Model into CUDA (RTX 4050)
        base_model = AutoModelForCausalLM.from_pretrained(
            BASE_MODEL,
            torch_dtype=torch.float16,
            device_map="auto"
        )

        # Inject Custom LoRA Adapters
        model = PeftModel.from_pretrained(
            base_model,
            ADAPTER_PATH,
            torch_dtype=torch.float16,
```

```
            device_map="auto"
        )
        print("✅ Astro-Ai Server is Ready on GPU!")
    except Exception as e:
        print(f"Error loading model: {e}")
    yield
app = FastAPI(lifespan=lifespan)
class ChatRequest(BaseModel):
    prompt: str
    max_tokens: int = 1024
@app.post("/generate")
async def generate_text(request: ChatRequest):
    inputs = tokenizer(request.prompt, return_tensors="pt").to(model.device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=request.max_tokens,
            temperature=0.7,
            repetition_penalty=1.1
        )

    response_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return {"response": response_text}
```

Appendix B: Sample Output (Synthetic Multilingual Dataset)

The following is a sample JSON output extracted from the Astro-Ai data generation pipeline, demonstrating the system's ability to format educational content accurately in regional languages.

```
json
{
  "id": "sample_hi_001",
  "language": "hindi",
```

"topic": "Science",
"context": "प्रकाश संश्लेषण (Photosynthesis) वह प्रक्रिया है जिसके द्वारा हरे पौधे सूर्य के प्रकाश, पानी और कार्बन डाइऑक्साइड का उपयोग करके अपना भोजन बनाते हैं। इस प्रक्रिया में ऑक्सीजन गैस बाहर निकलती है।",
"question": "पौधे अपना भोजन बनाने के लिए किस गैस का उपयोग करते हैं?",
"answers": {
"text": ["कार्बन डाइऑक्साइड"],
"answer_start": [83]
},
"source_url": "astro-ai-local-gen"
}